

FinaScore SA — Documentation des Jeux de Données

DHI Academy · Data & AI Engineering Program · Projet 4 · Mois 4

Dr. Michel MAMA TOULOU & Ing Franklin KASSAN'NGA

Contexte

FinaScore SA est une fintech camerounaise de scoring crédit pour les PME et micro-entrepreneurs de la zone CEMAC (6 pays). Son API traite 50 000 demandes/jour pour 280 institutions partenaires.

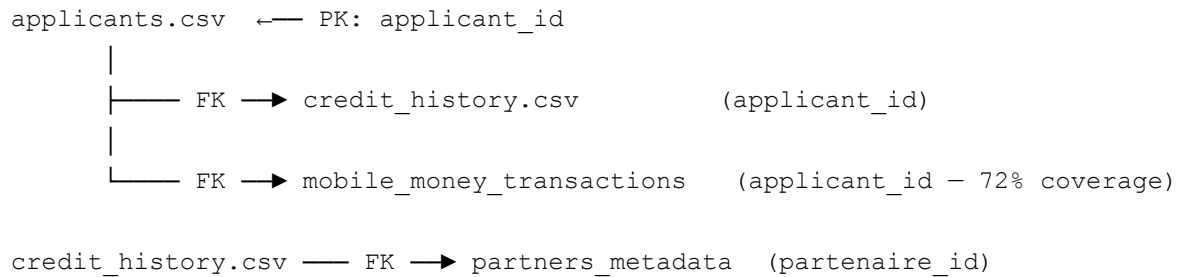
Ces 4 fichiers CSV constituent la base d'entraînement du système de scoring crédit ML du **Projet 4**.

 **Données synthétiques** à usage pédagogique uniquement. Les anomalies présentes (valeurs manquantes, doublons, formats inconsistants) sont **intentionnelles** et reproduisent les conditions réelles d'un projet terrain.

Fichiers

Fichier	Lignes	Colonnes	Rôle
applicants.csv	51 000	13	Profils demandeurs + TARGET
credit_history.csv	101 913	9	Historique remboursements
mobile_money_transactions.csv	36 000	7	Comportement mobile money (72%)
partners_metadata.csv	280	6	Référentiel partenaires

Schéma relationnel



Dictionnaire des données

1. applicants.csv — Profils des demandeurs

Colonne	Type	Valeurs	Anomalies	Description
applicant_id	str	FIN-XXXX-YYYY	~2% doublons	Identifiant unique. Les doublons = re-soumissions.
age	int	[14–75]	257 valeurs < 18	Âge en années. Valeurs < 18 = erreurs de saisie.
pays	str	6 pays CEMAC	19 variantes	CMR, GAB, COG, CAF, TCD, GNQ. Casse et abréviations variables.
secteur_activite	str	5 secteurs	15 variantes	commerce, agriculture, artisanat, services, élevage.
revenu_mensuel_xaf	float	[-3M ; 3M]	NULL 3% · Négatifs 1%	Revenu mensuel déclaré en FCFA. Négatifs = erreurs.
anciennete_emploi	float	[0–120]	NULL 1%	Ancienneté en mois. 0 = informel / démarrage.
ratio_endettement	float	[0.00–3.00]	NULL 1%	Ratio dette/revenu. Valeurs > 1 = surendettement (légitimes).

historique_credit	float	[300–850]	NULL 18%	Score [300–850]. NULL = primo-demandeur sans historique.
nb_credits_actifs	int	[0–8]	Aucune	Nombre de crédits en cours auprès des partenaires.
mobile_money_score	float	[2.4–99.8]	NULL 28%	Score comportement MM [0–100]. NULL = pas de compte/consentement.
zone	str	3 zones	11 variantes	urbain, periurbain, rural. Libellés inconsistants.
date_demande	str	2022–2025	Formats multiples	ISO, FR (dd/mm/yyyy), tirt, timestamp Unix.
defaut_paiement 	int	0 / 1	Taux : 12.1%	TARGET. 0 = solvable, 1 = défaut. Déséquilibre 88%/12%.

Stats clés :

- Taux de défaut : **12.1%** (déséquilibre → ne pas utiliser l'Accuracy)
- Demandeurs uniques : 50 000 (51 000 avec doublons)
- Pays dominant : CMR (41.6%)

2. credit_history.csv — Historique de remboursement

Colonne	Type	Valeurs	Anomalies	Description
applicant_id	str	FK	—	Référence vers applicants.csv.
credit_id	str	CRED-XXXXX	—	Identifiant du crédit.
montant_xaf	float	[50k–5M]	—	Montant en FCFA. Médiane ≈ 150 000 FCFA.
duree_mois	int	3,6,9,12,18,24,36	—	Durée contractuelle. Mode = 12 mois.

nb_retards	int	[0–7]	—	Mensualités payées en retard.
jours_retard_max	float	[0–119]	NULL 1.5%	Retard maximal en jours. NULL = info non remontée.
statut_final	str	12 variantes	Labels inconsistants	Canoniques : rembourse, défaut, en_cours, restructure.
partenaire_id	str	PTN-0001... 0280	FK	Institution ayant accordé le crédit.
date_credit	str	2020–2025	Formats multiples	ISO, FR, tiret.

Stats clés :

- 101 913 crédits pour ~40 200 demandeurs
- Montant moyen : 401 000 FCFA
- 12 variantes de statut_final à standardiser

3. mobile_money_transactions.csv — Comportement mobile money

⚠ **Couverture 72%** — 28% des demandeurs n'ont pas de données MM.

Colonne	Type	Valeurs	Anomalies	Description
applicant_id	str	FK	1 ligne/demandeur	Clé étrangère vers applicants.
nb_transactions_mois	float	[2–59]	—	Transactions mensuelles moyennes.
volume_entrant_xaf	float	[300–1.1M]	NULL 1%	Volume mensuel entrant en FCFA.
volume_sortant_xaf	float	[200–1M]	NULL 1%	Volume mensuel sortant en FCFA.

<code>regularite_score</code>	float	[0.1–100]	NULL 2%	Régularité des flux [0–100].
<code>operateur</code>	str	4 + variantes	11 variantes	MTN, ORANGE, CAMTEL, mixte.
<code>anciennete_compte_mois</code>	int	[1–84]	—	Ancienneté du compte MM en mois.

4. `partners_metadata.csv` — Référentiel partenaires

Table propre, sans anomalies intentionnelles.

Colonne	Type	Valeurs	Description
<code>partenaire_id</code>	str	PTN-0001...0280	Clé primaire.
<code>nom</code>	str	Texte	Nom de l'institution.
<code>type</code>	str	4 modalités	microfinance (44%), banque (25%), mobile_money (21%), cooperative (11%).
<code>pays</code>	str	6 codes CEMAC	Pays standardisé (pas de bruit).
<code>seuil_score</code>	int	[440–718]	Score minimum accepté. Moyenne : 572 pts.
<code>volume_mensuel</code>	int	[21–1185]	Demandes/mois.

Guide de nettoyage

```
import pandas as pd, numpy as np

df = pd.read_csv("applicants.csv")

# 1. Supprimer les doublons (garder la plus récente)
df = df.sort_values("date_demande").drop_duplicates("applicant_id", keep="last")
```

```

# 2. Filtrer les mineurs
df = df[df["age"] >= 18]

# 3. Corriger les revenus négatifs
df.loc[df["revenu_mensuel_xaf"] < 0, "revenu_mensuel_xaf"] = np.nan

# 4. Standardiser les pays
pays_map = {
    "cameroun": "CMR", "cmr": "CMR", "Cameroun": "CMR",
    "gab": "GAB", "Gabon": "GAB", "cog": "COG", "Congo": "COG", "congo": "COG",
    "caf": "CAF", "RCA": "CAF", "tcd": "TCD", "Tchad": "TCD", "tchad": "TCD",
    "gnq": "GNQ", "Guinee Eq.": "GNQ",
}
df["pays"] = df["pays"].replace(pays_map)

# 5. Créer les flags informatifs
df["flag_primo"] = df["historique_credit"].isna().astype(int)
df["flag_no_mm"] = df["mobile_money_score"].isna().astype(int)

# 6. IMPUTATIONS → dans Pipeline() uniquement (pas ici !)

```

Démarrage rapide

```

from sklearn.pipeline      import Pipeline
from sklearn.preprocessing import RobustScaler, OneHotEncoder
from sklearn.impute        import SimpleImputer
from sklearn.compose       import ColumnTransformer
from xgboost               import XGBClassifier

NUM_COLS = ["revenu_mensuel_xaf", "ratio_endettement", "anciennete_emploi",
            "historique_credit", "nb_credits_actifs", "mobile_money_score"]
FLAG_COLS = ["flag_primo", "flag_no_mm"]
CAT_COLS = ["pays", "secteur_activite", "zone"]

preproc = ColumnTransformer([
    ("num", Pipeline([("imp", SimpleImputer(strategy="median")),
                     ("sc", RobustScaler())])), NUM_COLS),
    ("cat", Pipeline([("imp", SimpleImputer(strategy="most_frequent")),
                     ("enc", OneHotEncoder(handle_unknown="ignore"))])), CAT_COLS),
    ("flg", "passthrough", FLAG_COLS),
])

pipe = Pipeline([

```

```

("prep", preproc),
("model", XGBClassifier(n_estimators=300, max_depth=6,
                        learning_rate=0.05, scale_pos_weight=7.3)),
])
pipe.fit(X_train, y_train)
# → AUC-ROC attendu ≈ 0.86

```

Performances attendues

Modèle	AUC-ROC	F1-Score
Ridge (baseline)	0.741 ± 0.012	0.621
Random Forest	0.821 ± 0.008	0.724
XGBoost	0.863 ± 0.006	0.781
XGBoost + Optuna	0.877 ± 0.005	0.802

Objectif minimum soutenance : AUC-ROC ≥ 0.80 en cross-validation 5-fold

FAQ rapide

Question	Réponse
Pourquoi 12% de défauts ?	Réaliste pour le crédit semi-formel CEMAC. Déséquilibre intentionnel.
historique_credit NULL = ?	Primo-demandeur. Créer flag_primo, imputer dans Pipeline.
Supprimer les NaN ?	Non. Imputer dans Pipeline(). Créer des flags.
StandardScaler ou RobustScaler ?	RobustScaler — résistant aux outliers de revenus.
scale_pos_weight XGBoost ?	88/12 ≈ 7.3 — obligatoire pour les classes déséquilibrées.

